

Sectoral API — Stock Screener Tag & Returns Tracker

Project Documentation

Version: 1.0

Date: June 27, 2026

Author: Ujain

Status: Approved for Implementation

Table of Contents

1. [Project Overview](#)
 2. [Technology Stack](#)
 3. [System Architecture](#)
 4. [Data Extraction from Chartlink](#)
 5. [Database Design](#)
 6. [AI-Powered Sector Tagging \(Gemini\)](#)
 7. [Redis Caching Strategy](#)
 8. [Weekly Returns Calculation](#)
 9. [API Endpoints](#)
 10. [Intraday Webhook Alerts](#)
 11. [Scheduler & Cron Jobs](#)
 12. [Project Structure](#)
 13. [Deployment Guide \(Render\)](#)
 14. [Scaling Considerations](#)
 15. [Verification & Testing Plan](#)
 16. [Appendix](#)
-

1. Project Overview

1.1 Problem Statement

Indian equity market investors use screeners on Chartlink to identify stocks matching specific technical or fundamental criteria. However, the raw screener output lacks:

- **Semantic classification** — What industry/sector does the stock belong to at a granular level?
- **Performance tracking** — How did the screened stocks perform over the following week?
- **Tag-based analytics** — Which industry sectors (as a cohort) are delivering the best returns?

1.2 Solution

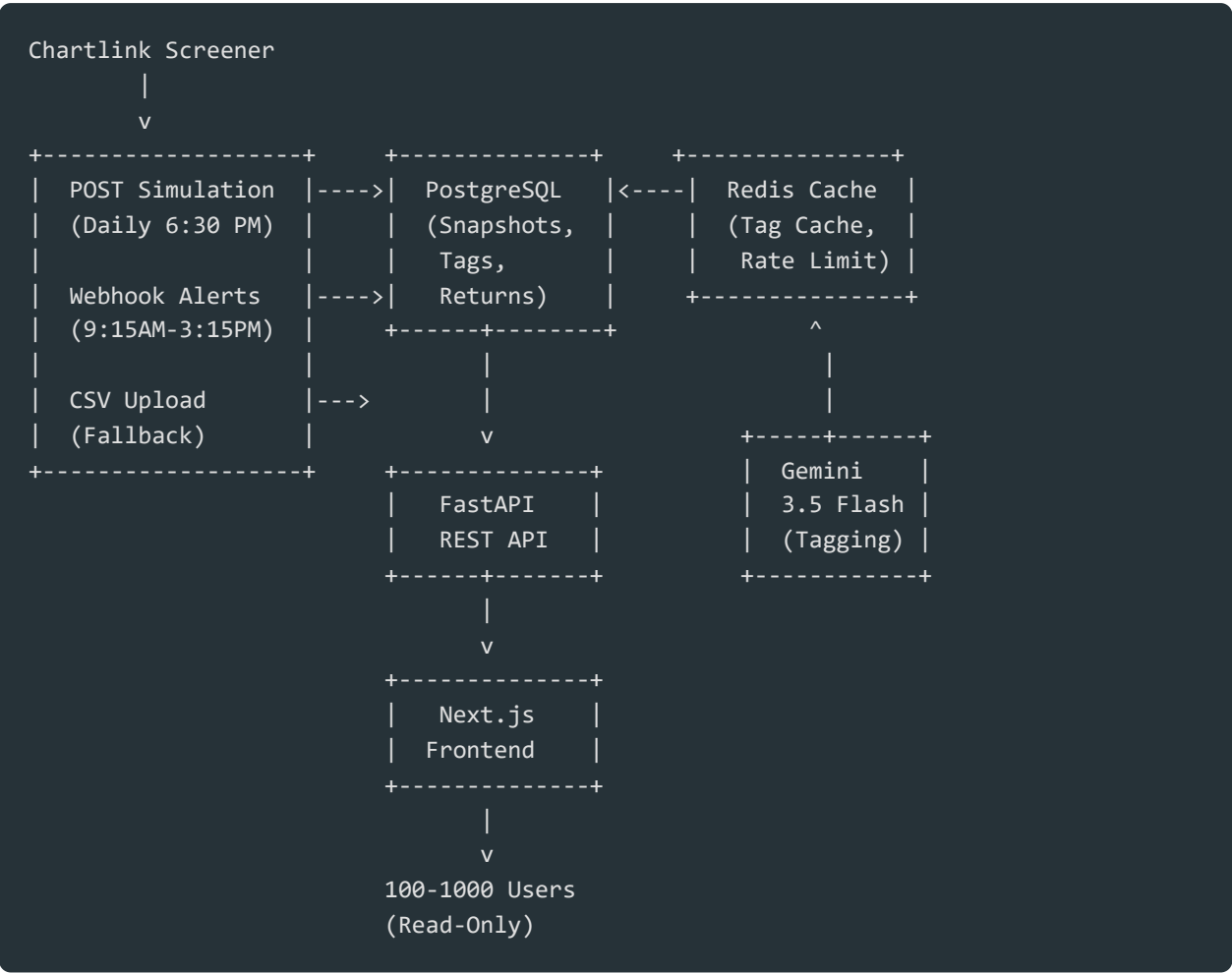
Build a Python backend system that:

1. **Ingests** daily stock screener data from Chartlink (automated POST simulation + webhook alerts + CSV fallback)
2. **Tags** each stock with up to 3 AI-generated sector/industry labels using the Gemini 3.5 Flash free-tier API
3. **Caches** tags in Redis to avoid redundant API calls
4. **Calculates** weekly LTP returns for individual stocks and tag-based cohorts
5. **Serves** the data via a FastAPI REST API consumed by a basic Next.js frontend
6. **Alerts** users when tracked stocks hit configured metrics during market hours (via Chartlink webhooks)

1.3 Target Scale

- **100 to 1,000 users** (read-only API consumers)
- **100 to 500 unique stocks** per screener per day
- Users do **not** write data — only server-side cron jobs and webhooks write to the database

1.4 High-Level Flow



2. Technology Stack

2.1 Backend

Layer	Technology	Version	Purpose
Language	Python	3.12+	Core runtime
API Framework	FastAPI	Latest	Async REST API with auto Swagger docs
ASGI Server	Uvicorn	Latest	Production-grade ASGI server
Database	PostgreSQL	16	Primary data store
ORM	SQLModel	Latest	SQLAlchemy + Pydantic hybrid models
Migrations	Alembic	Latest	Database schema versioning
Cache	Redis	7+	Tag caching + rate limiting
AI Model	Gemini 3.5 Flash	Free Tier	Sector/industry tag generation
HTTP Client	httpx	Latest	Async HTTP for scraping + API calls
Scheduler	APScheduler	Latest	In-process cron job scheduler
Data Validation	Pydantic v2	Latest	Request/response schema validation

2.2 Frontend

Layer	Technology	Purpose
Framework	Next.js	Basic dashboard UI
Language	TypeScript	Type-safe frontend code
Styling	CSS	Dashboard styling

2.3 Infrastructure

Component	Service	Plan
Backend Hosting	Render	Free tier (web service)
Frontend Hosting	Render	Free tier (static/web)
PostgreSQL	Render Managed	Free tier
Redis	Render Managed or Upstash	Free tier
External Cron	cron-job.org	Free (to keep Render awake)

2.4 AI Model Decision Rationale

The following models were evaluated for sector/industry tag generation:

Model	Access Method	Speed	JSON Output Quality	Free Tier	Verdict
Gemini 3.5 Flash	Google AI Studio API	Fastest	Excellent	~10-15 RPM, ~1500 RPD	Selected
Gemini 3.1 Flash-Lite	Google AI Studio API	Fast	Good	Shared limits	Backup option
Gemma 4 (API)	Google AI Studio API	Fast	Good	Same shared limits	No advantage over Gemini 3.5
Gemma 4 (Self-hosted)	Own GPU	Varies	Good	No API cost, hardware cost	Overkill for this scale
Gemini 2.0 Flash	Deprecated	—	—	—	Do not use

Why Gemini 3.5 Flash over Gemma 4?

- Gemini 2.0 Flash was **deprecated on June 1, 2026**. Do not use it.
- Gemma 4 is available via the same Google AI Studio API, meaning it **shares the same free-tier quota** as Gemini 3.5 Flash. There is no advantage to using Gemma via API.
- Self-hosting Gemma 4 requires a GPU server (the 12B model needs ~16GB VRAM). This only makes sense for data privacy or extremely high volume — neither applies at 100–1000 users with under 500 stocks/day.
- Gemini 3.5 Flash is the **newest, fastest, and best at following structured JSON output prompts** — exactly what we need for tag generation.

2.5 Database Decision: Why Not a Time-Series Database?

Factor	PostgreSQL (Plain)	TimescaleDB (PG Extension)	InfluxDB / QuestDB
Data volume at our scale	~500 stocks x 1 row/day = ~180K rows/year	Designed for millions+ rows	Designed for billions of data points
Query pattern	Simple lookups: "LTP for stock X on dates Y-Z"	Optimized for: "50-day moving avg across 10K stocks"	Optimized for high-frequency tick data
Setup on Render	Native managed PostgreSQL	Requires custom Docker	Separate service, added complexity
ORM compatibility	Full SQLAlchemy support	Hypertables require some raw SQL	No standard ORM support
Migration path	Start here, add TimescaleDB later	One command to enable	Completely different system

Verdict: At our scale (~500 rows/day, daily granularity, 1-week lookback), plain PostgreSQL is optimal. TimescaleDB can be added later with a single SQL command if we ever scale to intraday tick data:

```
CREATE EXTENSION timescaledb;
SELECT create_hypertable('daily_snapshots', 'snapshot_date');
```

No data migration needed — the existing table is converted in place.

3. System Architecture

3.1 Component Diagram



3.2 Data Flow Summary

Flow	Trigger	Source	Destination	Frequency
Screener fetch	APScheduler cron	Chartlink	PostgreSQL	Daily 6:30 PM IST (weekdays)
Tag generation	Post-screener hook	Gemini API	PostgreSQL + Redis	Per new/changed stock
Webhook alert	Chartlink push	Chartlink	PostgreSQL	Hourly, 9:15 AM–3:15 PM IST
Weekly returns	APScheduler cron	PostgreSQL (read)	PostgreSQL (write)	Every Monday 7:00 AM IST
API reads	User request	PostgreSQL + Redis	JSON response	On-demand

4. Data Extraction from Chartlink

Chartlink has **no official public API**. Three extraction layers are implemented in priority order.

4.1 Layer 1: POST Simulation (Primary — Daily Bulk Fetch)

Schedule: Monday–Friday, 6:30 PM IST (after market close)

How it works:

1. Send a `GET` request to `https://chartink.com` to extract the CSRF token from the HTML `<meta>` tag
2. Send a `POST` request to `https://chartink.com/screener/process` with:
3. Header: `X-CSRF-TOKEN: <extracted_token>`
4. Body: `scan_clause=<your_technical_scan_formula>`
5. Receive JSON response containing stock data: symbol, name, LTP, volume, change%, etc.
6. Parse and store in the `daily_snapshots` table

Implementation details:


```

class ChartlinkScraper:
    BASE_URL = "https://chartink.com"
    PROCESS_URL = f"{BASE_URL}/screener/process"

    async def fetch_csrf_token(self) -> str:
        """GET the homepage and extract CSRF token from meta tag."""
        response = await self.client.get(self.BASE_URL)
        # Parse: <meta name="csrf-token" content="...">
        return extracted_token

    async def run_screener(self, scan_clause: str) -> list[dict]:
        """POST scan_clause to /screener/process, return stock list."""
        token = await self.fetch_csrf_token()
        response = await self.client.post(
            self.PROCESS_URL,
            data={"scan_clause": scan_clause},
            headers={"X-CSRF-TOKEN": token}
        )
        return response.json()["data"] # List of stock dicts

```

Rate limiting: 2-second delay between consecutive screener requests to avoid overloading Chartlink.

4.2 Layer 2: Webhook Alerts (Secondary — Intraday Event Triggers)

Schedule: 7 triggers per day during Indian market hours (9:15 AM – 3:15 PM IST), spaced 1 hour apart.

Trigger #	Time (IST)
1	9:15 AM (Market Open)
2	10:15 AM
3	11:15 AM
4	12:15 PM
5	1:15 PM
6	2:15 PM
7	3:15 PM

How it works:

1. Configure screener alerts in Chartlink's UI with a webhook URL pointing to:
<https://sectoral-api.onrender.com/webhook/chartlink>
2. Set alert conditions (e.g., "Stock crosses 200 DMA", "RSI > 70", etc.)

3. When conditions are met, Chartlink POSTs JSON payload to the webhook
4. Backend checks if the stock exists in the database and is being tracked
5. If tracked: save alert event + trigger notification
6. If not tracked: acknowledge receipt, do nothing

Payload format (expected from Chartlink):

```
{
  "scan_name": "200 DMA Crossover",
  "scan_url": "https://chartink.com/screener/...",
  "alert_name": "My Alert",
  "triggered_at": "2026-06-27 10:15:00",
  "stocks": [
    {
      "nsecode": "RELIANCE",
      "bsecode": "500325",
      "per_chg": "1.25",
      "close": "2850.50",
      "volume": "12500000"
    }
  ]
}
```

4.3 Layer 3: CSV Upload (Fallback)

When to use: If POST simulation breaks (Chartlink adds CAPTCHA, changes HTML structure, etc.)

How it works:

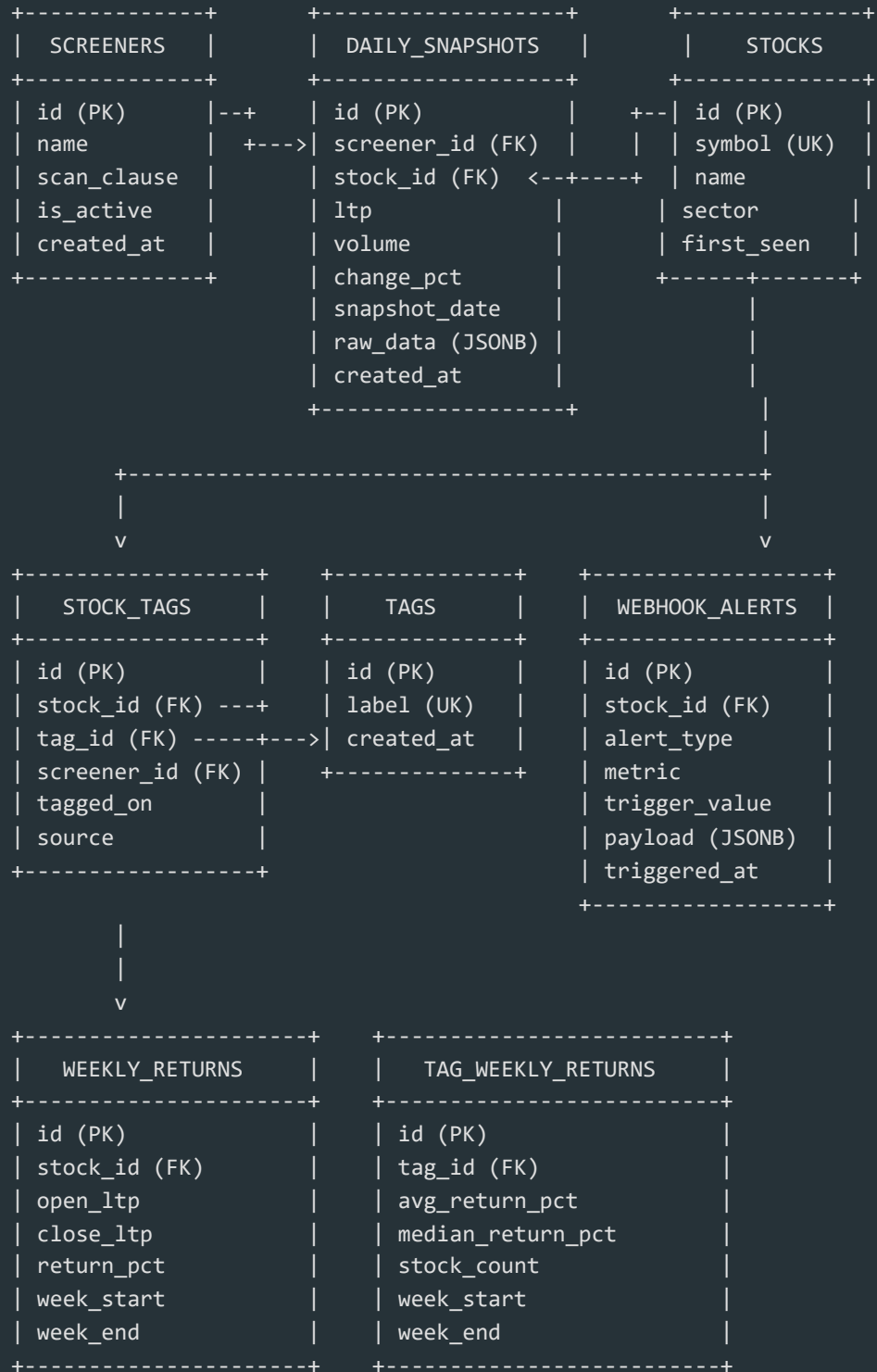
1. User manually exports CSV from Chartlink's screener page ("Download CSV" button)
2. Uploads via `POST /api/screener/upload` endpoint
3. Backend parses CSV columns: Symbol, Name, LTP, Volume, Change%
4. Data enters the same pipeline as Layer 1 (save snapshot, tag, compute returns)

Accepted CSV format:

```
Sr,Symbol,Name,LTP,Volume,Change%
1,RELIANCE,Reliance Industries,2850.50,12500000,1.25
2,TCS,Tata Consultancy Services,3420.00,5600000,0.85
```

5. Database Design

5.1 Entity Relationship Diagram



5.2 Table Definitions

SCREENERS

Stores screener configurations. Each screener has a `scan_clause` (Chartlink formula).

Column	Type	Constraints	Description
id	INTEGER	PK, AUTO	Unique identifier
name	VARCHAR(255)	NOT NULL	Human-readable screener name
scan_clause	TEXT	NOT NULL	Chartlink scan formula
is_active	BOOLEAN	DEFAULT TRUE	Whether to run this screener daily
created_at	TIMESTAMP	DEFAULT NOW	Record creation time

STOCKS

Master stock table. Each stock is uniquely identified by its NSE/BSE symbol.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTO	Unique identifier
symbol	VARCHAR(50)	UNIQUE, NOT NULL	NSE/BSE stock symbol
name	VARCHAR(255)	NOT NULL	Company name
sector	VARCHAR(100)	NULLABLE	Broad sector (from Chartlink if available)
first_seen	TIMESTAMP	DEFAULT NOW	When this stock first appeared

DAILY_SNAPSHOTS

One row per stock per screener per day. Stores the point-in-time market data.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTO	Unique identifier
stock_id	INTEGER	FK -> stocks.id	Stock reference
screeener_id	INTEGER	FK -> screeners.id	Which screener found this stock
ltp	DECIMAL(12,2)	NOT NULL	Last Traded Price
volume	BIGINT	NULLABLE	Trading volume
change_pct	DECIMAL(8,4)	NULLABLE	Daily change percentage
snapshot_date	DATE	NOT NULL	Market date
raw_data	JSONB	NULLABLE	Full Chartlink response for this stock
created_at	TIMESTAMP	DEFAULT NOW	Record insertion time

Composite unique constraint: (stock_id, screener_id, snapshot_date) — prevents duplicate entries.

TAGS

Normalized tag table. Each unique sector/industry label is stored once.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTO	Unique identifier
label	VARCHAR(100)	UNIQUE, NOT NULL	Lowercase tag label (e.g., "auto ancillaries")
created_at	TIMESTAMP	DEFAULT NOW	Tag creation time

STOCK_TAGS

Junction table linking stocks to tags, with provenance tracking.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTO	Unique identifier
stock_id	INTEGER	FK -> stocks.id	Stock reference
tag_id	INTEGER	FK -> tags.id	Tag reference
screener_id	INTEGER	FK -> screeners.id	Which screener context generated this tag
tagged_on	DATE	NOT NULL	Date tag was assigned
source	VARCHAR(50)	DEFAULT 'gemini'	Source: 'gemini' or 'manual'

Composite unique constraint: (stock_id, tag_id) — a stock-tag pair is unique.

WEEKLY_RETURNS

Pre-computed weekly return for each stock. Materialized by the Monday cron job.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTO	Unique identifier
stock_id	INTEGER	FK -> stocks.id	Stock reference
open_ltp	DECIMAL(12,2)	NOT NULL	LTP at week start
close_ltp	DECIMAL(12,2)	NOT NULL	LTP at week end
return_pct	DECIMAL(8,4)	NOT NULL	((close - open) / open) x 100
week_start	DATE	NOT NULL	Monday of the week
week_end	DATE	NOT NULL	Friday of the week

TAG_WEEKLY_RETURNS

Aggregated weekly return for all stocks sharing a tag. Materialized by the Monday cron job.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTO	Unique identifier
tag_id	INTEGER	FK -> tags.id	Tag reference
avg_return_pct	DECIMAL(8,4)	NOT NULL	Mean return across stocks with this tag
median_return_pct	DECIMAL(8,4)	NOT NULL	Median return across stocks with this tag
stock_count	INTEGER	NOT NULL	Number of stocks in this cohort
week_start	DATE	NOT NULL	Monday of the week
week_end	DATE	NOT NULL	Friday of the week

WEBHOOK_ALERTS

Stores intraday alert events received from Chartlink webhooks.

Column	Type	Constraints	Description
id	INTEGER	PK, AUTO	Unique identifier
stock_id	INTEGER	FK -> stocks.id	Stock reference
alert_type	VARCHAR(100)	NOT NULL	e.g., "200_DMA_CROSSOVER"
metric	VARCHAR(100)	NULLABLE	e.g., "200 DMA", "RSI"
trigger_value	DECIMAL(12,4)	NULLABLE	Value at which alert fired
payload	JSONB	NULLABLE	Full webhook payload
triggered_at	TIMESTAMP	NOT NULL	When the alert triggered

5.3 Database Indexes

Optimized for read-heavy workloads (100-1000 users querying, only cron jobs writing):

```
-- Fast weekly lookups
CREATE INDEX idx_snapshots_stock_date
  ON daily_snapshots (stock_id, snapshot_date);

-- Fast tag queries
CREATE INDEX idx_stock_tags_stock
  ON stock_tags (stock_id);
CREATE INDEX idx_stock_tags_tag
  ON stock_tags (tag_id);

-- Fast return queries
CREATE INDEX idx_weekly_returns_stock_week
  ON weekly_returns (stock_id, week_start);
CREATE INDEX idx_tag_returns_tag_week
  ON tag_weekly_returns (tag_id, week_start);

-- Alert history
CREATE INDEX idx_alerts_stock_time
  ON webhook_alerts (stock_id, triggered_at DESC);
```

6. AI-Powered Sector Tagging (Gemini)

6.1 Tagging Philosophy

Tags classify stocks by their **business domain and industry sub-sector** — not by market behavior or trading patterns.

Correct tag examples: - "auto ancillaries", "EV components", "two-wheelers" - "waste management", "water treatment", "renewable energy" - "private banking", "insurance", "NBFCs" - "IT services", "SaaS", "digital payments" - "specialty chemicals", "agrochemicals", "pharmaceuticals" - "cement", "steel", "infrastructure"

Incorrect tags (explicitly excluded): - "momentum play", "breakout candidate", "value pick", "high dividend"

6.2 Prompt Template

```
TAGGING_PROMPT = """You are a stock market sector classifier for the Indian equity market.
Given a stock's name, symbol, and available data, generate exactly 3 tags that describe
the BUSINESS DOMAIN and INDUSTRY of this company.
```

```
Tags should be specific industry sub-sectors like:
```

- "auto ancillaries", "EV components", "two-wheelers"
- "waste management", "water treatment", "renewable energy"
- "private banking", "insurance", "NBFCs"
- "IT services", "SaaS", "digital payments"
- "specialty chemicals", "agrochemicals", "pharmaceuticals"
- "cement", "steel", "infrastructure"

```
Do NOT generate tags about stock behavior (no "momentum", "breakout", "value pick").
Return ONLY a JSON array of exactly 3 lowercase strings.
```

```
Stock: {name} ({symbol})
Sector: {sector}
LTP: Rs.{ltp} | Change: {change_pct}%
Screener: {screener_name}
"""
```

6.3 Example Input/Output

Input:

```
Stock: Motherson Sumi Wiring India Ltd (MSUMI)
Sector: Auto
LTP: Rs.72.50 | Change: 2.10%
Screener: Weekly Breakout Stocks
```

Output:

```
["auto ancillaries", "wiring harness", "EV components"]
```


6.4 Tag Generation Service Logic

```

async def generate_tags_for_stock(stock, screener, db, redis):
    cache_key = f"stock:tags:{stock.symbol}:{screener.id}"

    # Step 1: Check Redis cache
    cached = await redis.get(cache_key)
    if cached:
        return json.loads(cached) # Cache HIT -- skip everything

    # Step 2: Check DB for existing tags
    existing_tags = await get_stock_tags(stock.id, screener.id, db)

    # Step 3: Determine if re-tagging is needed
    if existing_tags:
        last_snapshot = await get_latest_snapshot(stock.id, db)
        ltp_change = abs(stock.ltp - last_snapshot.ltp) / last_snapshot.ltp
        if ltp_change < 0.05: # Less than 5% change
            # Stock hasn't moved significantly -- cache existing and return
            await redis.setex(cache_key, 7 * 86400, json.dumps(existing_tags))
            return existing_tags

    # Step 4: Call Gemini API
    prompt = TAGGING_PROMPT.format(
        name=stock.name, symbol=stock.symbol,
        sector=stock.sector or "Unknown",
        ltp=stock.ltp, change_pct=stock.change_pct,
        screener_name=screener.name
    )
    new_tags = await call_gemini(prompt) # Returns ["tag1", "tag2", "tag3"]

    # Step 5: Normalize and deduplicate
    new_tags = [tag.strip().lower() for tag in new_tags]

    # Step 6: Upsert into DB
    await upsert_stock_tags(stock.id, new_tags, screener.id, db)

    # Step 7: Cache in Redis (TTL: 7 days)
    await redis.setex(cache_key, 7 * 86400, json.dumps(new_tags))

    return new_tags

```

6.5 Rate Limiting

Gemini free tier limits (~10-15 RPM, ~1500 RPD) are managed by a token-bucket rate limiter:

```
class GeminiRateLimiter:
    def __init__(self, rpm=10, rpd=1400):
        self.rpm = rpm
        self.rpd = rpd
        self.minute_tokens = rpm
        self.day_tokens = rpd

    async def acquire(self):
        """Wait until a token is available."""
        while self.minute_tokens <= 0:
            await asyncio.sleep(6) # Wait ~6s (60s / 10 RPM)
            self.minute_tokens -= 1
            self.day_tokens -= 1
```

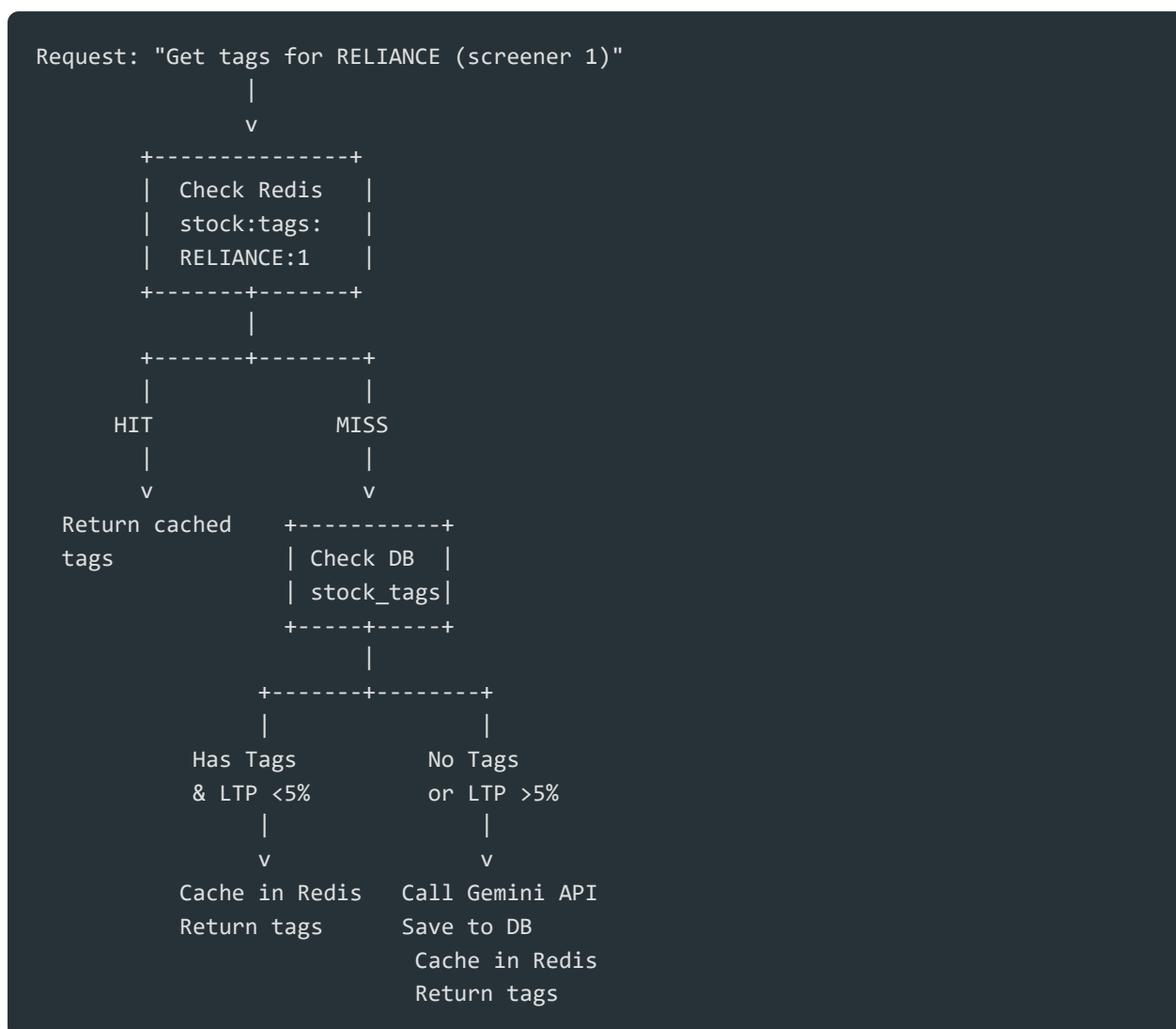
On `429 RESOURCE_EXHAUSTED`, exponential backoff is applied: 2s, 4s, 8s, 16s, then fail.

7. Redis Caching Strategy

7.1 Cache Keys

Key Pattern	Value	TTL	Purpose
<code>stock:tags:{symbol}:{screener_id}</code>	JSON array of 3 tag strings	7 days	Skip Gemini API if tags exist
<code>gemini:rate:{minute_bucket}</code>	Request count	60 seconds	Rate limit tracking
<code>gemini:rate:daily</code>	Request count	24 hours	Daily quota tracking

7.2 Cache Flow Diagram



7.3 Cache Invalidation

- **TTL-based:** Tags auto-expire after 7 days, forcing re-evaluation
- **On significant change:** If a stock's LTP changes >5% from its last snapshot, tags are regenerated regardless of cache state
- **Manual flush:** Admin endpoint `DELETE /api/cache/tags/{symbol}` to force re-tagging

8. Weekly Returns Calculation

8.1 Per-Stock Weekly Return

Schedule: Every Monday, 7:00 AM IST

Formula:

```
return_pct = ((close_ltp - open_ltp) / open_ltp) x 100
```

Where: - `open_ltp` = LTP from the earliest `daily_snapshot` in the previous week (Monday or first trading day) - `close_ltp` = LTP from the latest `daily_snapshot` in the previous week (Friday or last trading day)

Implementation:

```
async def calculate_weekly_returns(db):
    today = date.today()
    week_end = today - timedelta(days=(today.weekday() + 2) % 7) # Last Friday
    week_start = week_end - timedelta(days=4) # Previous Monday

    stocks = await get_all_active_stocks(db)

    for stock in stocks:
        open_snap = await get_earliest_snapshot(stock.id, week_start, week_end)
        close_snap = await get_latest_snapshot(stock.id, week_start, week_end)

        if open_snap and close_snap and open_snap.ltp > 0:
            return_pct = ((close_snap.ltp - open_snap.ltp) / open_snap.ltp) * 100
            await save_weekly_return(
                stock_id=stock.id,
                open_ltp=open_snap.ltp,
                close_ltp=close_snap.ltp,
                return_pct=round(return_pct, 4),
                week_start=week_start,
                week_end=week_end,
                db=db
            )
```

8.2 Per-Tag Aggregated Return

After computing individual stock returns, aggregate by tag:

```

async def calculate_tag_weekly_returns(db, week_start, week_end):
    tags = await get_all_tags(db)

    for tag in tags:
        # Get all stocks linked to this tag
        stock_ids = await get_stock_ids_for_tag(tag.id, db)

        # Get their weekly returns
        returns = await get_weekly_returns_for_stocks(
            stock_ids, week_start, week_end, db
        )

        if returns:
            return_values = [r.return_pct for r in returns]
            await save_tag_weekly_return(
                tag_id=tag.id,
                avg_return_pct=round(mean(return_values), 4),
                median_return_pct=round(median(return_values), 4),
                stock_count=len(return_values),
                week_start=week_start,
                week_end=week_end,
                db=db
            )

```

8.3 Example Output

Stock Weekly Returns:

Stock	Open LTP	Close LTP	Return %	Week
RELIANCE	Rs.2,820.00	Rs.2,850.50	+1.08%	Jun 23-27
TCS	Rs.3,380.00	Rs.3,420.00	+1.18%	Jun 23-27
MSUMI	Rs.70.00	Rs.72.50	+3.57%	Jun 23-27

Tag Weekly Returns:

Tag	Avg Return	Median Return	# Stocks	Week
auto ancillaries	+2.33%	+2.10%	12	Jun 23-27
IT services	+0.95%	+1.05%	8	Jun 23-27
waste management	+4.10%	+3.80%	5	Jun 23-27

9. API Endpoints

9.1 Stock APIs

Method	Endpoint	Description	Response
GET	/api/stocks	List all tracked stocks with latest LTP and tags	Paginated stock list
GET	/api/stocks/{symbol}	Single stock details	Stock with tags + latest snapshot
GET	/api/stocks/{symbol}/history	Daily snapshot history	Paginated list of snapshots
GET	/api/stocks/{symbol}/returns	Weekly returns for a stock	List of weekly return records

Example: GET /api/stocks/RELIANCE

```
{
  "symbol": "RELIANCE",
  "name": "Reliance Industries Ltd",
  "sector": "Energy",
  "tags": ["oil & gas", "petrochemicals", "telecom"],
  "latest_snapshot": {
    "ltp": 2850.50,
    "change_pct": 1.25,
    "volume": 12500000,
    "date": "2026-06-27"
  }
}
```

9.2 Tag APIs

Method	Endpoint	Description	Response
GET	/api/tags	List all tags with stock counts	Tag list
GET	/api/tags/{label}/stocks	Stocks under a specific tag	Stock list filtered by tag
GET	/api/tags/{label}/returns	Weekly returns for a tag cohort	Tag return history

Example: GET /api/tags/auto%20ancillaries/returns

```
{
  "tag": "auto ancillaries",
  "returns": [
    {
      "week_start": "2026-06-23",
      "week_end": "2026-06-27",
      "avg_return_pct": 2.33,
      "median_return_pct": 2.10,
      "stock_count": 12
    },
    {
      "week_start": "2026-06-16",
      "week_end": "2026-06-20",
      "avg_return_pct": 1.15,
      "median_return_pct": 0.98,
      "stock_count": 10
    }
  ]
}
```

9.3 Screener APIs

Method	Endpoint	Description	Request Body
POST	/api/screener/run	Manually trigger a screener run	{ "screener_id": 1 }
POST	/api/screener/upload	Upload CSV fallback	multipart/form-data with CSV file

9.4 Webhook APIs

Method	Endpoint	Description	Caller
POST	/webhook/chartlink	Receive Chartlink alert payload	Chartlink
GET	/api/alerts	List recent alert events	Frontend
GET	/api/alerts?stock={symbol}	Alerts filtered by stock	Frontend

9.5 Dashboard APIs

Method	Endpoint	Description
GET	/api/dashboard	Summary: top gaining stocks, top tags, recent alerts

Example: GET /api/dashboard

```
{
  "top_stocks_this_week": [
    { "symbol": "MSUMI", "return_pct": 3.57, "tags": ["auto ancillaries"] }
  ],
  "top_tags_this_week": [
    { "tag": "waste management", "avg_return_pct": 4.10, "stock_count": 5 }
  ],
  "recent_alerts": [
    {
      "stock": "RELIANCE",
      "alert_type": "200_DMA_CROSSOVER",
      "time": "2026-06-27T10:15:00"
    }
  ],
  "total_stocks_tracked": 342,
  "total_tags": 87
}
```

9.6 Cache Management APIs

Method	Endpoint	Description
DELETE	/api/cache/tags/{symbol}	Force re-tag a stock (clears Redis cache)
DELETE	/api/cache/flush	Flush all tag caches (admin only)

10. Intraday Webhook Alerts

10.1 Alert Flow

```

Chartlink Alert Fires (e.g., RELIANCE crosses 200 DMA at 10:15 AM)
|
v
POST /webhook/chartlink
|
v
Parse payload --> Extract stock symbol(s)
|
v
+--- Is stock in our DB? ---+
|                               |
YES                             NO
|                               |
v                               v
Save to webhook_alerts      Log & ignore
table                      Return 200 OK
|
v
Trigger notification
(future: push notification,
email, or WebSocket)

```

10.2 Alert Configuration

Alerts are configured directly in Chartlink's web UI:

1. Go to your screener on Chartlink
2. Click "Create Alert"
3. Set the webhook URL: `https://sectoral-api.onrender.com/webhook/chartlink`
4. Choose scan interval: **1 hour**
5. Set market hours: **9:15 AM to 3:30 PM IST**

10.3 Alert Types (Examples)

Alert Type	Metric	Description
200_DMA_CROSSOVER	200 DMA	Stock price crosses above 200-day moving average
RSI_OVERBOUGHT	RSI > 70	RSI indicates overbought condition
VOLUME_SPIKE	Volume > 2x avg	Unusual volume activity
52_WEEK_HIGH	52-week high	Stock hits a new 52-week high

11. Scheduler & Cron Jobs

11.1 Job Schedule

Job Name	Schedule	Days	Description
daily_screener_fetch	6:30 PM IST	Mon-Fri	Fetch all active screeners, save snapshots, generate tags
weekly_returns_calc	7:00 AM IST	Monday	Compute weekly returns for all stocks and tags
cleanup_old_snapshots	12:00 AM IST	Sunday	Archive/delete snapshots older than 90 days

11.2 APScheduler Configuration

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from apscheduler.triggers.cron import CronTrigger

scheduler = AsyncIOScheduler(timezone="Asia/Kolkata")

# Daily screener fetch (Mon-Fri, 6:30 PM IST)
scheduler.add_job(
    daily_screener_fetch,
    CronTrigger(hour=18, minute=30, day_of_week="mon-fri"),
    id="daily_screener_fetch",
    name="Daily Screener Fetch",
    replace_existing=True
)

# Weekly returns calculation (Monday, 7:00 AM IST)
scheduler.add_job(
    weekly_returns_calc,
    CronTrigger(hour=7, minute=0, day_of_week="mon"),
    id="weekly_returns_calc",
    name="Weekly Returns Calculator",
    replace_existing=True
)

# Cleanup old data (Sunday, midnight IST)
scheduler.add_job(
    cleanup_old_snapshots,
    CronTrigger(hour=0, minute=0, day_of_week="sun"),
    id="cleanup_old_snapshots",
    name="Cleanup Old Snapshots",
    replace_existing=True
)
```

11.3 Render Free Tier Caveat

Render's free tier web services spin down after 15 minutes of inactivity. This means APScheduler jobs will not fire if the service is asleep.

Workaround: Use an external cron service (free) to keep the service alive:

Service	What It Does
cron-job.org	Free. Sends HTTP request to your /api/health every 14 minutes
UptimeRobot	Free. Pings your service URL every 5 minutes

Alternative: Use Render's Cron Jobs feature (paid, ~\$1/job/month) for guaranteed execution.

12. Project Structure

```

sectoral_api/
|
+-- backend/
|   +-- app/
|       |   +-- __init__.py
|       |   +-- main.py                # FastAPI app, lifespan events, router mounting
|       |   +-- config.py              # Pydantic Settings: env vars, DB URL, API keys
|       |   +-- database.py            # Async SQLAlchemy engine, session factory
|       |   +-- redis_client.py        # Redis connection manager
|       |   |
|       |   +-- models/                # SQLAlchemy database models
|       |       |   +-- __init__.py
|       |       |   +-- stock.py       # Stock, DailySnapshot
|       |       |   +-- tag.py         # Tag, StockTag
|       |       |   +-- returns.py    # WeeklyReturns, TagWeeklyReturns
|       |       |   +-- screener.py   # Screener
|       |       |   +-- webhook.py    # WebhookAlert
|       |       |
|       |       +-- schemas/           # Pydantic request/response schemas
|       |           |   +-- __init__.py
|       |           |   +-- stock.py
|       |           |   +-- tag.py
|       |           |   +-- returns.py
|       |           |
|       |       +-- routers/           # FastAPI route handlers
|       |           |   +-- __init__.py
|       |           |   +-- stocks.py  # Stock CRUD, history, returns
|       |           |   +-- tags.py    # Tag listing, tag returns
|       |           |   +-- screener.py # Screener run, CSV upload
|       |           |   +-- returns.py # Weekly returns, dashboard
|       |           |   +-- webhooks.py # Chartlink webhook receiver
|       |           |
|       |       +-- services/          # Business logic layer
|       |           |   +-- __init__.py
|       |           |   +-- chartlink_scraper.py # POST simulation, CSRF extraction, CSV parser
|       |           |   +-- gemini_tagger.py    # Gemini API calls, tag caching, Redis integrati
|       |           |   +-- returns_calculator.py # Weekly stock + tag return computation
|       |           |   +-- webhook_handler.py  # Alert processing, notification dispatch
|       |           |   +-- scheduler.py        # APScheduler job definitions
|       |           |
|       |       +-- utils/             # Shared utilities
|       |           |   +-- __init__.py
|       |           |   +-- rate_limiter.py     # Token bucket rate limiter for Gemini
|       |           |
|       |       +-- alembic/           # Database migrations
|       |           |   +-- env.py
|       |           |   +-- versions/         # Migration scripts
|       |           |
|       |       +-- tests/             # Test suite
|       |           |   +-- test_chartlink_scraper.py

```

```
| | +-- test_gemini_tagger.py
| | +-- test_returns_calculator.py
| | +-- test_webhook_handler.py
| | +-- test_api_endpoints.py
| |
| +-- alembic.ini           # Alembic configuration
| +-- requirements.txt      # Python dependencies
| +-- pyproject.toml       # Project metadata
| +-- .env                 # Environment variables (not committed)
|
+-- frontend/             # Next.js application
| +-- src/
| | +-- app/
| | | +-- page.tsx         # Dashboard home page
| | | +-- stocks/page.tsx  # Stock listing page
| | | +-- tags/page.tsx    # Tag listing + returns page
| | | +-- layout.tsx       # Root layout
| | +-- components/
| | | +-- StockTable.tsx   # Stock data table component
| | | +-- TagCard.tsx      # Tag summary card component
| | | +-- ReturnsChart.tsx # Returns chart component
| +-- package.json
| +-- next.config.js
|
+-- render.yaml           # Render Blueprint (infrastructure-as-code)
+-- README.md             # Project README
```

13. Deployment Guide (Render)

13.1 Services Overview

Service	Type	Plan	Runtime
sectoral-api	Web Service	Free	Python
sectoral-frontend	Web Service	Free	Node.js
sectoral-db	PostgreSQL	Free	Managed
sectoral-redis	Redis	Free	Managed (or Upstash)

13.2 Render Blueprint (render.yaml)

```
services:
  - type: web
    name: sectoral-api
    runtime: python
    plan: free
    buildCommand: pip install -r backend/requirements.txt
    startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
    rootDir: backend
    envVars:
      - key: DATABASE_URL
        fromDatabase:
          name: sectoral-db
          property: connectionString
      - key: REDIS_URL
        value: <from Upstash or Render Redis>
      - key: GEMINI_API_KEY
        sync: false

  - type: web
    name: sectoral-frontend
    runtime: node
    plan: free
    buildCommand: npm install && npm run build
    startCommand: npm start
    rootDir: frontend
    envVars:
      - key: NEXT_PUBLIC_API_URL
        value: https://sectoral-api.onrender.com

databases:
  - name: sectoral-db
    plan: free
    postgresMajorVersion: 16
```

13.3 Environment Variables

Variable	Source	Example
DATABASE_URL	Render PostgreSQL	postgresql://user:pass@host:5432/sectoral_db
REDIS_URL	Render Redis or Upstash	redis://default:pass@host:6379
GEMINI_API_KEY	Google AI Studio	Alza...
ENVIRONMENT	Manual	production
CORS_ORIGINS	Manual	https://sectoral-frontend.onrender.com

13.4 Deployment Steps

1. Push code to a GitHub repository
2. Connect the repository to Render
3. Render auto-detects render.yaml and provisions all services
4. Set the GEMINI_API_KEY secret in Render dashboard
5. Run Alembic migrations: `alembic upgrade head`
6. Configure external cron (cron-job.org) to ping /api/health every 14 minutes
7. Set up Chartlink webhook URLs to point to <https://sectoral-api.onrender.com/webhook/chartlink>

14. Scaling Considerations

14.1 Current Architecture Capacity

Dimension	Capacity	Bottleneck at
API read throughput	1,000+ concurrent readers	Render free tier limits (single instance)
Database writes	~500 rows/day (cron only)	Not a concern
Gemini API calls	~1,500/day (free tier)	~500 unique stocks/day with caching
Redis operations	~10,000/day	Not a concern
Storage	~180K snapshot rows/year	Render free PG: 1GB limit

14.2 Scaling Path (When Needed)

Trigger	Action
> 1,000 users causing slow API	Upgrade Render to paid tier (\$7/mo) for multiple instances
> 500 stocks/day hitting Gemini limit	Batch prompts (5 stocks per prompt = 5x fewer calls)
Database > 1GB	Upgrade Render PostgreSQL (paid) or enable compression
Need intraday tick data	Add TimescaleDB extension to existing PostgreSQL
Need real-time updates	Add WebSocket support via FastAPI

14.3 Key Design Decisions for Scale

1. **Pre-computed tables:** weekly_returns and tag_weekly_returns are materialized by cron — no on-the-fly calculation for API reads
 2. **Read-only users:** All 100-1000 users hit pre-computed endpoints; no user-triggered writes
 3. **Smart caching:** Redis prevents redundant Gemini calls; tags are stable (industry doesn't change daily)
 4. **Single data pipeline:** Screener data is fetched once by the server, not per-user
-

15. Verification & Testing Plan

15.1 Automated Tests

```
# Unit tests
pytest tests/test_chartlink_scraper.py      # Mock HTTP, verify CSRF extraction + JSON pa
pytest tests/test_gemini_tagger.py          # Mock Gemini API, verify tag caching logic
pytest tests/test_returns_calculator.py    # Verify return percentage math
pytest tests/test_webhook_handler.py       # Verify alert processing for tracked/untrack

# Integration tests
pytest tests/test_api_endpoints.py         # Full API integration with test database
```


15.2 Manual Verification Checklist

#	Test	Expected Result
1	Run screener manually via POST /api/screener/run	daily_snapshots table populated with stock data
2	Check generated tags	Tags should be sector/industry (e.g., "auto ancillaries"), NOT market behavior
3	Re-run same stocks (no significant LTP change)	Redis cache hit — no Gemini API call made (check logs)
4	Trigger Chartlink webhook for a tracked stock	Alert saved in webhook_alerts table
5	Trigger Chartlink webhook for an untracked stock	200 OK returned, no database write
6	Wait for Monday cron, check weekly_returns	Return % matches manual calculation
7	Check tag_weekly_returns	Aggregated avg/median returns are mathematically correct
8	Deploy to Render, hit all API endpoints	All endpoints return valid JSON responses
9	Load test with 50 concurrent readers	Response time < 500ms for dashboard endpoint
10	Let service sleep on Render, check external cron wakes it	APScheduler jobs fire on schedule

16. Appendix

16.1 Python Dependencies (requirements.txt)

```
fastapi>=0.115.0
uvicorn[standard]>=0.34.0
sqlmodel>=0.0.22
sqlalchemy[asyncio]>=2.0.0
asyncpg>=0.30.0
alembic>=1.14.0
httpx>=0.28.0
redis[hiredis]>=5.2.0
apscheduler>=3.11.0
google-genai>=1.14.0
pydantic-settings>=2.7.0
python-multipart>=0.0.18
pandas>=2.2.0
pytest>=8.3.0
pytest-asyncio>=0.25.0
```

16.2 Key API Rate Limits Reference

Service	Limit	Mitigation
Gemini 3.5 Flash (Free)	~10-15 RPM, ~1500 RPD	Redis caching, skip unchanged stocks
Chartlink (Unofficial)	No official limit	2-second delay between requests
Render Free Tier	750 hours/month, spins down after 15 min	External cron ping every 14 min

16.3 Glossary

Term	Definition
LTP	Last Traded Price — the most recent price at which a stock was traded
DMA	Day Moving Average — average closing price over N days
RSI	Relative Strength Index — momentum oscillator (0-100 scale)
CSRF	Cross-Site Request Forgery — security token required by Chartlink
RPM / RPD	Requests Per Minute / Requests Per Day — API rate limit units
Scan Clause	Chartlink's proprietary formula syntax for defining screener criteria
Hypertable	TimescaleDB's partitioned table optimized for time-series data
TTL	Time To Live — duration before a Redis cache entry expires
NBFC	Non-Banking Financial Company
NSE	National Stock Exchange of India
BSE	Bombay Stock Exchange

Document generated on June 27, 2026. This document serves as the complete technical specification for the Sectoral API project.